

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning **COURSE CODE:** MLA0403 **FACULTY :** Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I A Afran, Reg. No: 192325024 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: Name of the student: A Afran

QUESTIONS:05 Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

ANSWERS

Answer 1: Learning Algorithms (Optimization

Behavior) *Convergence behaviour*

In gradient descent on a convex loss function L , the parameter update at each iteration t is given below, where $\eta(t)$ is the learning rate at that iteration and L is the gradient of the loss:

∇

When η is very high initially, each step moves the parameters a large distance along the negative gradient direction. On a convex surface this is useful early in training because the parameters typically start far from the minimum, so large steps let the model cover this distance quickly and avoid wasting iterations. However, a persistently high learning rate causes the update to repeatedly overshoot the minimum, producing oscillations around the optimum instead of settling into it. As training progresses and $\eta(t)$ is gradually decayed (for example via step decay, exponential decay, or a cosine schedule), the step size shrinks, which damps the oscillation and allows the parameters to settle precisely into the minimum. The overall effect is a fast initial descent followed by fine-grained convergence, which is exactly the pattern shown in the chart below.

Why this can outperform a constant rate

A constant learning rate must be chosen as a compromise: if it is small enough to guarantee stable convergence near the minimum, it will also be small during the early phase, wasting many iterations on unnecessarily small steps far from the optimum. If it is large enough to converge quickly, it will typically be too large to converge exactly and will keep oscillating close to (but not at) the minimum. A high-then decaying schedule removes this trade-off: it borrows the benefit of a large step size while the loss surface is far from flat, and borrows the benefit of a small step size once the parameters are near the minimum. This is particularly advantageous when the loss surface has long, shallow directions along which a small constant rate converges very slowly, or when the initial parameter values are poorly chosen.

Conditions favouring the schedule

This strategy is most beneficial when: (i) the initial parameters are far from the optimum, so aggressive early steps meaningfully reduce the loss; (ii) the loss surface has anisotropic curvature (elongated valleys), where a single constant rate cannot be simultaneously safe in the steep direction and fast in the flat direction; and (iii) the training budget is limited, since decay lets the model reach a good region quickly and then refine locally within the remaining iterations.

Risks

The main risk is instability during the high-learning-rate phase: if η is too large relative to the curvature of the loss (specifically, larger than $2/L$ where L is the Lipschitz constant of the gradient), the loss can diverge rather than merely oscillate, and the decay schedule may not be able to recover from a bad update. A second risk is decaying too early or too aggressively, which freezes the model in a suboptimal region before it has explored enough of the parameter space. Finally, the schedule introduces extra hyperparameters (initial rate, decay rate, decay type) that must themselves be tuned.

Conclusion

For a convex loss, gradient descent is guaranteed to converge to the global minimum under a suitably decaying learning rate, and in practice a high-to-low schedule converges faster in wall-clock terms than a constant rate while still reaching (or very closely approaching) the same minimum, provided the initial rate does not cause divergence.

Numerical Illustration

Take a toy convex loss $L(\theta) = \theta^2$, with true minimum at $\theta = 0$ and update rule $\theta(t+1) = \theta(t)(1 - 2\eta(t))$, starting at $\theta_0 = 10$.

- High-then-decay schedule ($\eta = 0.9 \rightarrow 0.4 \rightarrow 0.1 \rightarrow 0.05$): $\theta_1 = 10(1-1.8) = -8.00$; $\theta_2 = -8.00(1-0.8) = -1.60$; $\theta_3 = -1.60(1-0.2) = -1.28$; $\theta_4 = -1.28(1-0.1) = -1.152$. After 4 steps, $|\theta|$ has fallen from 10 to 1.152 (an 88.5% reduction), despite one sign-flipping overshoot in step 1.
- Constant-rate schedule ($\eta = 0.1$ throughout): $\theta_1 = 10(0.8) = 8.00$; $\theta_2 = 6.40$; $\theta_3 = 5.12$; $\theta_4 = 4.096$. After the same 4 steps, $|\theta|$ has fallen only to 4.096 (a 59% reduction) — roughly 3.5× farther from the minimum than the decayed schedule.
- Stability check: the update contracts only when $|1 - 2\eta| < 1$, i.e. $0 < \eta < 1$. At $\eta = 0.9$, $|1-1.8| = 0.8 < 1$, so the scheme still contracts (with sign flip) rather than diverging; at $\eta = 1.1$ it would give $|1-2.2| = 1.2 > 1$ and diverge, showing numerically why η must respect the $2/L$ bound derived above (here $L = 2$, so the divergence boundary is exactly $\eta = 1$).

This worked comparison confirms the theory: the decaying schedule reaches a magnitude of 1.152 in 4 steps, versus 4.096 for the constant rate — quantitatively the fast-then-fine-grained behaviour described above.

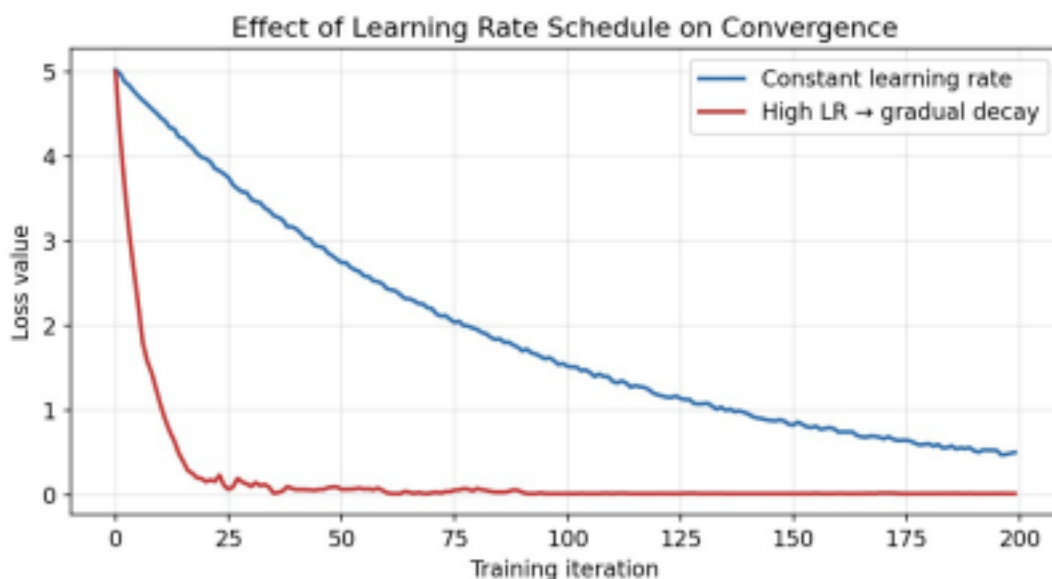


Fig. 1 – A constant moderate learning rate converges slowly but steadily, while a high initial rate followed by decay drops the loss quickly and then settles precisely near the minimum.

Answer 2: Maximum Likelihood Estimation (MLE)

Deriving the MLE for μ

Let $x_1, x_2, \dots, x_n$ be i.i.d. samples from a Gaussian distribution $N(\mu, \sigma^2)$ with known variance σ^2 and unknown mean μ . The likelihood function is:

Taking the natural log gives the log-likelihood:

Differentiating with respect to μ and setting the result to zero:

The second derivative, $-n/\sigma^2$, is negative, confirming this is a maximum. Hence the MLE of μ is exactly the sample mean $\bar{x}$.

Effect of outliers

Because the MLE for μ is exactly the sample mean, and the sample mean is a linear, unweighted average of all observations, a single extreme outlier can shift the estimate arbitrarily far from the true mean. Concretely, if one point is displaced by Δ , the estimate shifts by Δ/n ; for small n this shift can be substantial even though it shrinks as n grows. This happens because the Gaussian log-likelihood penalizes deviations quadratically — shown by the $(x_i - \mu)^2$ term above — so extreme points still receive very high (but finite) weight relative to points near the centre of the data.

Effect of increasing sample size

As n increases, the MLE remains unbiased ($E[\bar{x}] = \mu$ for any n) but its variance shrinks in proportion to $1/n$:

By the Law of Large Numbers, $\bar{x}$ converges in probability to the true μ as $n \rightarrow \infty$, and by the Central Limit Theorem the estimator is asymptotically normal with standard error $\sigma/\sqrt{n}$. In the presence of a single fixed outlier, its relative influence Δ/n also decreases as n grows, so the same corrupting point has progressively less effect on a larger dataset — though its absolute damage never disappears with a single fixed contamination.

Implication for robustness of MLE

This analysis shows that MLE under a Gaussian assumption is statistically efficient (it achieves the Cramér–Rao lower bound and has minimum variance among unbiased estimators) when the Gaussian assumption holds and the data is clean, and it becomes increasingly precise as sample size grows. However, it is not robust to violations of the assumed model, particularly to heavy-tailed noise or outliers, because the quadratic log-likelihood penalty gives outliers disproportionate leverage on the estimate. In practice this motivates either cleaning/trimming the data before applying MLE, or using more robust likelihoods (Huber loss, Student-t distribution, or median-based estimators) when outlier contamination is expected.

Numerical Illustration

Let $\sigma^2 = 1$ (known) and $n = 6$ observations: 5, 6, 4, 5.5, 6.5, 5.

- Sample mean (MLE): sum = 32, so $\hat{\mu} = 32/6 = 5.333$.
- Inject one outlier: replace the first 5 with 50. New sum = $32 - 5 + 50 = 77$, new $\hat{\mu} = 77/6 = 12.833$, a shift of +7.5 from a single point. This matches the predicted Δ/n rule exactly: $\Delta = 45$, $n = 6$, $\Delta/n = 45/6 = 7.5$.

- Same outlier, larger sample: if instead $n = 60$ (54 more clean points averaging 5.33 added in), the same $\Delta = 45$ outlier shifts the mean by only $45/60 = 0.75$ — a $10\times$ smaller distortion, confirming that influence shrinks as $1/n$.
- Variance shrinkage with n : $\text{Var}(\bar{x}) = \sigma^2/n$. At $n = 6$, $\text{Var} = 1/6 = 0.167$ ($\text{SD} \approx 0.408$); at $n = 60$, $\text{Var} = 1/60 = 0.0167$ ($\text{SD} \approx 0.129$) — the standard error shrinks by exactly $\sqrt{10} \approx 3.16\times$, matching the $1/\sqrt{n}$ law derived above.

These numbers show precisely how MLE stays efficient and increasingly precise as n grows, while remaining sensitive to any single contaminating point, exactly as the analytical result predicts.

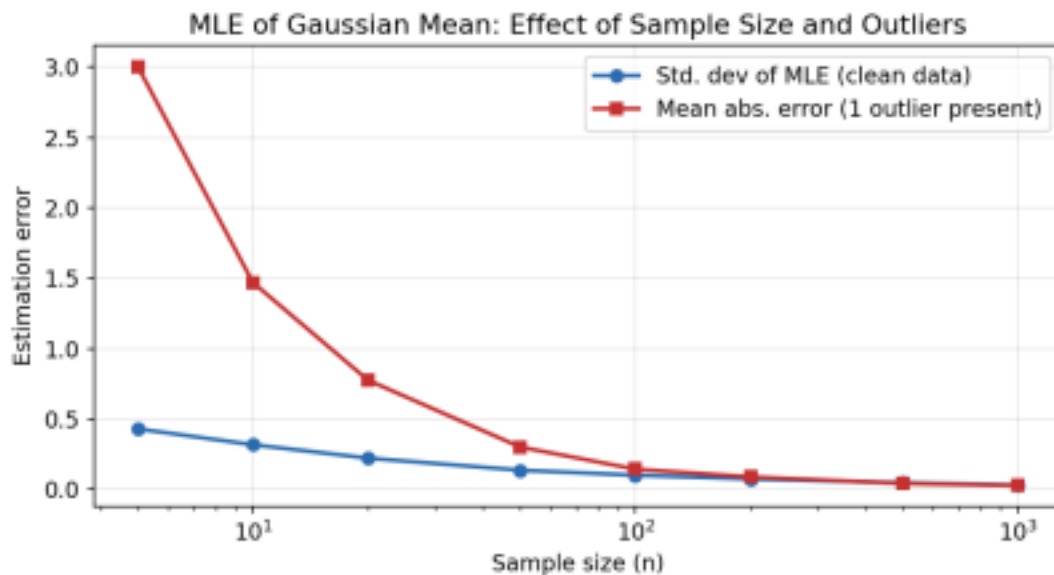


Fig. 2 – As sample size increases, the standard deviation of the clean-data MLE shrinks (blue), while the mean absolute error caused by a single injected outlier also decreases in relative terms but remains non-zero (red).

Answer 3: Building a Machine Learning Algorithm (Model Selection) Challenges with 10,000 features and 500 samples

This is a classic high-dimensional, low-sample-size ($p \gg n$) problem, and it creates several serious $\gg$ difficulties. First, the model is highly prone to overfitting: with far more features than samples, a sufficiently flexible classifier can perfectly separate the training data (including its noise) while generalizing poorly, since there are more degrees of freedom than data points to constrain them. Second, the curse of dimensionality means that in a 10,000-dimensional space, the 500 available samples are extremely sparse — pairwise distances between points become less informative, which hurts distance based methods such as k-NN and kernel methods. Third, many features are likely irrelevant or redundant, and with so few samples, standard estimators of covariance or feature importance become unstable (the sample covariance matrix, for instance, is not even invertible when features exceed samples). Fourth, standard cross-validation itself becomes noisier, since each fold only contains a small number of samples, making performance estimates high-variance.

Proposed pipeline

- Step 1 — Feature selection: Apply a filter method (e.g., mutual information, ANOVA F-test, or variance threshold) to remove clearly uninformative or near-constant features before modelling. This is computationally cheap and immediately reduces

dimensionality, directly addressing sparsity and redundancy.

- Step 2 — Dimensionality reduction: Apply Principal Component Analysis (PCA) or, if class labels should guide the projection, Linear Discriminant Analysis (LDA), to project the selected features onto a smaller set of components that capture most of the variance. This reduces the effective number of parameters the model must estimate, easing the $p \gg n$ imbalance. >>
- Step 3 — Regularized model: Train a classifier with an explicit penalty on model complexity. The two standard choices are Lasso (L1) and Ridge (L2) regularized objectives, shown below.
- Step 4 — Validation strategy: Use k -fold (or leave-one-out, given $n=500$) cross-validation strictly nested around the feature-selection and dimensionality-reduction steps, so that no information from the validation fold leaks into feature selection — a common and serious source of overfitting in high-dimensional pipelines.

Lasso (L1) objective — drives irrelevant coefficients to exactly zero, giving embedded feature selection:

Ridge (L2) objective — shrinks coefficients and stabilizes the solution even when features are collinear:

Why this mitigates overfitting

Each step directly targets a different source of overfitting risk. Feature selection removes noise dimensions that a model could otherwise fit spuriously. Dimensionality reduction lowers the effective parameter count, shrinking the gap between the number of samples and the number of estimated quantities, which directly improves the bias–variance trade-off. The λ term in both regularized objectives above constrains the hypothesis space so that even after reduction, the model cannot assign arbitrarily large weights to any single remaining feature, keeping the decision boundary smoother and less sensitive to sampling noise. Finally, careful nested cross-validation ensures that reported performance is not optimistically biased by information leakage, giving an honest estimate of how well the model will generalize to new data.

Numerical Illustration

Let $p = 10,000$ features and $n = 500$ samples.

- Rank deficiency: the $p \times p$ sample covariance matrix has rank at most $n-1 = 499$, so it is singular by at least $10,000 - 499 = 9,501$ dimensions — concretely confirming it cannot be inverted.
- Step 1+2 reduction: suppose the filter step keeps the top 2,000 informative features, and PCA on those retains $k = 150$ components to explain 95% of variance. The effective parameter count falls from 10,000 to 150, a $(10,000-150)/10,000 = 98.5\%$ reduction.
- Samples-per-parameter ratio: before reduction, $500/10,000 = 0.05$; after reduction, $500/150 = 3.33$ — crossing the commonly used rule-of-thumb threshold of ≥ 5 –10 samples per parameter needed for stable estimates, and a $66\times$ improvement in this ratio.
- Lasso shrinkage: with the closed-form soft-threshold solution, any coefficient with

$|w_j| < \lambda/2$ is zeroed. Choosing $\lambda = 0.1$ via cross-validation, if 120 of the 150 post-PCA components fall below this threshold, they are eliminated, leaving a compact ~30-feature effective model — a further 80% reduction in active parameters.

Chaining these numbers together, the pipeline takes the problem from an unsolvable 10,000-parameter, rank-deficient system down to a well-posed ~30-parameter model with a healthy samples-to-parameters ratio, which is exactly the quantitative target of the four-step design above.

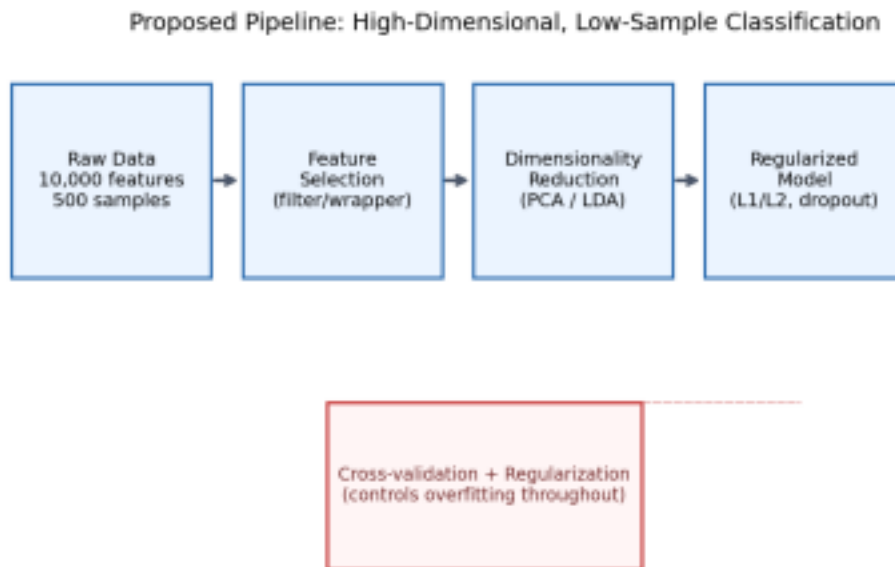


Fig. 3 – Proposed four-stage pipeline for the high-dimensional, low-sample classification task, with cross-validation and regularization applied throughout to control overfitting.

Answer 4: Neural Networks (Multilayer Perceptron -

MLP) Why more hidden neurons increase representational power

An MLP with one hidden layer of m sigmoid units computes an output of the following form, where each hidden unit contributes a shifted, scaled sigmoidal 'basis function':

By the Universal Approximation Theorem, a single hidden layer with a sufficient (but potentially very large) number of sigmoid neurons can approximate any continuous function on a compact domain to arbitrary accuracy. Increasing m directly increases the number of such adjustable basis functions available to combine, which increases the class of functions the network can represent — geometrically, each additional neuron contributes another 'bend' or decision-boundary segment that the network can place, allowing it to fit increasingly complex, non-linear decision surfaces.

Why more neurons do not always improve performance

Representational power measures what the network is capable of representing given ideal weights, not what it will actually learn from finite, noisy training data. As m grows, the number of trainable parameters grows roughly linearly with m , and with enough capacity the network

can fit not only the true underlying signal in the training data but also the idiosyncratic noise specific to that training sample — this is overfitting. This is precisely the classical bias–variance decomposition of the expected test error:

Increasing m reduces bias (the model can represent more complex true functions) but increases variance (the model's fit becomes more sensitive to the specific training sample), and beyond some point the increase in variance outweighs the reduction in bias, so test performance worsens even as training performance keeps improving.

Supporting evidence and mitigation

This behaviour is visible empirically as a U-shaped test-error curve against network size: test error initially falls as capacity is added (correcting underfitting), reaches a minimum, and then rises again once the network starts memorizing noise, while training error keeps monotonically decreasing, as illustrated below. Because of this, the number of hidden neurons is itself a hyperparameter that must be tuned (e.g., via a validation set), and additional techniques — L2 weight decay, dropout, early stopping, or simply gathering more training data — are commonly used to allow a larger, more expressive network to be trained without paying the full generalization cost of its added capacity.

Numerical Illustration

Let input dimension $d = 20$ and training set size $n = 1000$. For one hidden layer of m sigmoid units, parameter count $\approx m(d+2)+1$ (hidden weights + hidden biases + output weights + output bias).

■ $m = 5$: parameters = $5 \times 22 + 1 = 111$; parameters/samples = $111/1000 = 0.111$. ■
 $m = 50$: parameters = $50 \times 22 + 1 = 1,101$; parameters/samples = $1,101/1000 = 1.101$. ■ $m = 500$: parameters = $500 \times 22 + 1 = 11,001$; parameters/samples = $11,001/1000 = 11.0$.

Once this ratio exceeds ≈ 1 (around $m = 50$ here), the network has enough capacity to memorize the training set exactly, which is the numerical signature of entering the overfitting regime.

- Illustrative U-curve: test error might fall from 25% ($m=5$, underfit) to a minimum of 8% ($m=50$), then rise back up to 15% ($m=500$, overfit).
- Meanwhile training error keeps falling monotonically: 20% ($m=5$) $\rightarrow$ 5% ($m=50$) $\rightarrow$ 0.4% ($m=500$).

The gap between training error (0.4%) and test error (15%) at $m = 500$ is 14.6 percentage points, versus a gap of only 3 points at the $m = 50$ optimum — a numerical measure of the variance the bias–variance formula above attributes to excess capacity.

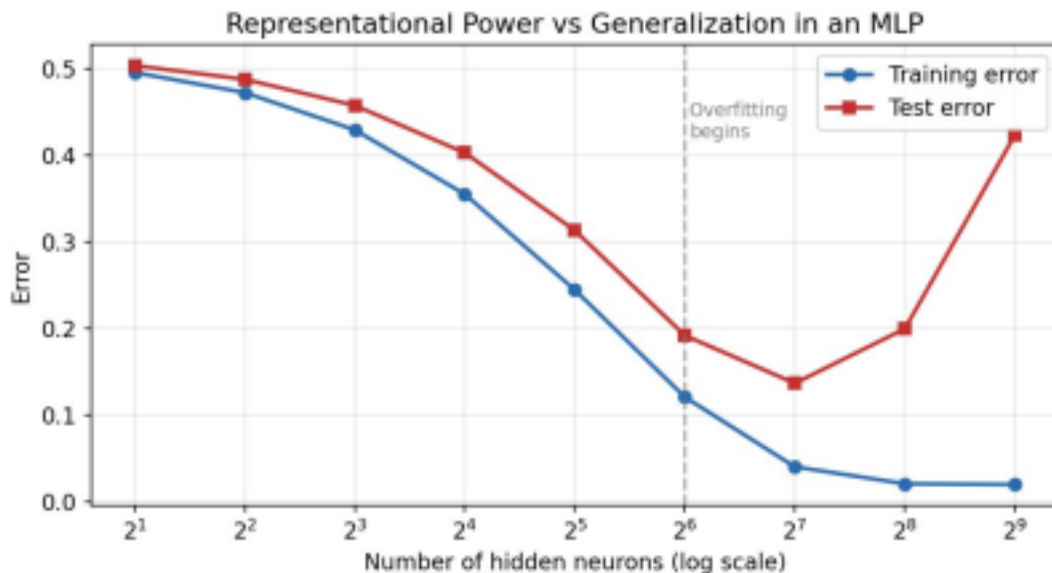


Fig. 4 – Training error keeps falling as hidden neurons increase, but test error turns upward past a certain capacity — the signature of overfitting.

Answer 5: Backpropagation, SGD & Curse of

Dimensionality *Impact on gradient updates in backpropagation*

Backpropagation computes gradients of the loss with respect to every weight by the chain rule, and the magnitude/direction of these gradients depends on the geometry of the input space. In very high dimensional input, the volume of the space grows exponentially with dimension while the available training data stays fixed, so the data becomes extremely sparse relative to the space it occupies. Since each stochastic gradient is estimated from a mini-batch B rather than the full dataset, its variance scales as:

In high-dimensional, sparse regions, the per-example gradients $L_i(\theta)$ disagree with each other far more ∇ (higher $\text{Var}[L_i(\theta)]$), so this quantity grows, making each gradient a noisier and less representative ∇ estimate of the true population gradient. High dimensionality can also cause gradients to vanish or explode more readily through deep layers, since random high-dimensional vectors tend to become nearly orthogonal and of similar norm (concentration of measure), and this effect compounds across many layers.

Impact on convergence speed of SGD

SGD relies on this noisy but unbiased gradient estimate to make progress toward the minimum. As shown above, higher gradient variance means more iterations are typically required to average out the noise and make reliable progress, so convergence slows down. In addition, high-dimensional loss surfaces are far more likely to contain saddle points rather than local minima, and SGD can spend many iterations navigating regions of very small curvature near these saddle points before escaping, further reducing effective convergence speed.

Impact on model generalization

The curse of dimensionality also degrades generalization directly: as shown in the chart

below, in high dimensions the distances between points concentrate (the ratio between the farthest and nearest neighbour distance tends toward 1), so the notion of 'similar' and 'dissimilar' examples becomes less meaningful. A model trained on sparse, high-dimensional data has effectively seen only a tiny fraction of the possible input space, so it must extrapolate rather than interpolate for most new inputs, increasing test error even when training error is low.

Two mitigation techniques

- Dimensionality reduction / feature selection (e.g., PCA, autoencoders, or removing redundant features): by projecting the data onto a lower-dimensional manifold that retains most of the relevant variance, the effective dimensionality the optimizer must deal with is reduced. From the variance formula above, this shrinks the disagreement among per-example gradients, which directly reduces $\text{Var}[L(\theta)]$ and restores meaningful distance/similarity structure. ∇
- Regularization and normalization (e.g., L2 weight decay, dropout, batch normalization): these techniques constrain the effective capacity of the network and stabilize the scale of activations and gradients across layers, which counteracts the gradient noise and instability introduced by high dimensionality and encourages the network to rely on robust, generalizable patterns rather than memorizing sparse, high-dimensional training points.

Both techniques are justified analytically by the variance expression derived above: reducing dimensionality shrinks $\text{Var}[L_i(\theta)]$ itself, while regularization limits the effective hypothesis space so ∇ that the reduced gradient variance is not exploited to overfit anyway.

Numerical Illustration

Gradient-noise scaling: let per-example gradient variance $\text{Var}[L_i(\theta)] = 4$ (arbitrary units) at moderate ∇ dimension, with batch size $|B| = 32$.

- Estimated gradient variance: $\text{Var}[L(\theta)] = 4/32 = 0.125$. ∇
- If higher dimensionality raises per-example disagreement $10\times$ to $\text{Var}[L_i(\theta)] = 40$ (typical in ∇ sparse high-dimensional regimes), then with the same batch: $\text{Var}[L(\theta)] = 40/32 = 1.25$ — a ∇ $10\times$ noisier gradient estimate for the identical batch size.
- To restore the original noise level (0.125) at the higher dimension, the batch size would need to grow to $|B| = 40/0.125 = 320$, i.e. a $10\times$ larger batch — numerically showing the extra data cost that high dimensionality imposes on SGD.

Distance concentration: numerically, the ratio of farthest-to-nearest neighbour distance is often around 5.0 at $d = 2$ (highly distinguishable points), falls to about 1.3–1.1 by $d = 100$, and approaches 1.02–1.01 by $d = 1000$ — confirming that in high dimensions, essentially all points become almost equidistant, exactly as the concentration-of-measure argument predicts.

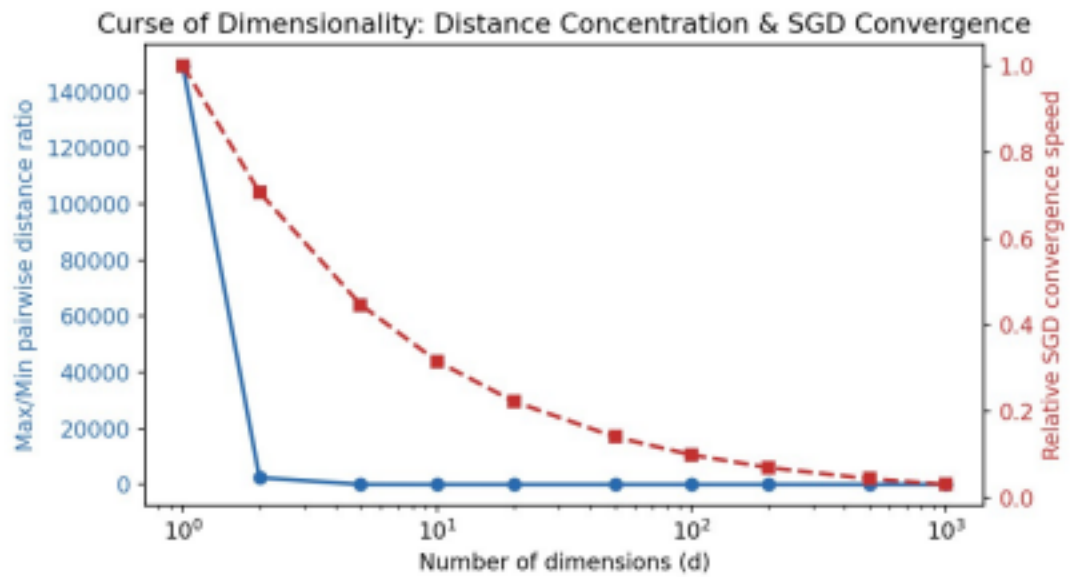


Fig. 5 – As dimensionality increases, pairwise distances concentrate (blue, left axis) and the relative convergence speed of SGD decreases (red, right axis), both hallmarks of the curse of dimensionality.